

# Tectopia: seguridad en una infraestructura Kubernetes autogestionada en producción

Informe final, IIC2531 Seguridad Computacional, 2026-1

Benjamín Saavedra

Joaquín Veloz

22 de junio de 2026

## Índice

|                                                    |    |
|----------------------------------------------------|----|
| 1. Introducción . . . . .                          | 1  |
| 2. Arquitectura del sistema en breve . . . . .     | 2  |
| 3. Modelo de amenaza . . . . .                     | 2  |
| 4. Mecanismos de seguridad implementados . . . . . | 3  |
| 5. Metodología y desarrollo . . . . .              | 7  |
| 6. Resultados . . . . .                            | 7  |
| 8. Discusión . . . . .                             | 9  |
| 9. Conclusiones . . . . .                          | 10 |
| 10. Material del Proyecto . . . . .                | 10 |

## 1. Introducción

*Tectopia* es una aplicación web multijugador en tiempo real, compuesta por un frontend en React, un backend en Node.js/Express con WebSockets vía Socket.IO, PostgreSQL como base de datos relacional y Redis como almacén de estado compartido entre réplicas (presencia, salas, adaptador de Socket.IO). A diferencia de un proyecto de curso típico, esta aplicación corre en producción real, sobre infraestructura autogestionada: cuatro máquinas virtuales sobre un servidor doméstico (Unraid), sin un proveedor de nube administrado de por medio. Esa decisión tiene una consecuencia directa para el alcance de este proyecto: el equipo es responsable no solo del código de la aplicación, sino también de gestionar la infraestructura del proyecto (k3s: una distribución liviana de Kubernetes), del aislamiento entre cargas de trabajo, de la gestión de secretos, de la cadena de CI/CD y de la disponibilidad del servicio. No hay una capa de la nube que absorba esas responsabilidades por defecto.

Esto hace que la seguridad de *Tectopia* sea un problema interesante más allá de lo habitual en una aplicación web. Primero, es una aplicación con estado mutable compartido (economía del juego, resultados de partidas, historial de chat) donde la *integridad* de ese estado importa tanto como la confidencialidad de los datos de usuario: un adversario que pueda falsificar su saldo o sus resultados rompe la propiedad central del sistema, no solo filtra información. Segundo, al usar WebSockets persistentes en un clúster con múltiples réplicas, aparecen problemas de seguridad y de confiabilidad que no existen en una API REST sin estado (afinidad de sesión, autenticación en el *handshake*, expulsión de sesiones activas entre réplicas). Tercero, el clúster aloja simultáneamente cargas de trabajo de **distinta confianza**: el ambiente de producción (*main*), un ambiente persistente compartido (*develop* y ramas *release/\**) y ambientes de *preview* efímeros que se crean automáticamente para **cualquier rama que se abra como pull request**, es decir, código que todavía no ha sido revisado propenso a fallas. Aislar correctamente estos tres niveles de confianza dentro de un mismo clúster es, en esencia, un problema de *multi-tenancy*.

El proyecto se propuso originalmente seis medidas concretas de seguridad sobre esta infraestructura: separación de privilegios vía contenedores/namespaces/pods de Kubernetes, detección de CVEs, autenticación y ACL de Redis, defensa contra XSS, un túnel de Cloudflare Zero Trust con TLS, y un “modelo de guardia”. El alcance real terminó siendo, en algunos frentes, más amplio que lo prometido (un sistema de RBAC completo dentro de la aplicación, auditoría a dos niveles, backups automatizados); en otros, más acotado y con brechas honestamente documentadas (el aislamiento llega hasta el namespace y la red, pero no baja al nivel de proceso dentro del contenedor ni a nivel de proceso por servicio).

**El objetivo principal de este proyecto para el curso fue auditar, evaluar y fortificar la infraestructura de Tectopia y su ciclo de vida, enfrentando los desafíos reales de aislar diferentes niveles de confianza (*multi-tenancy*) en un entorno autogestionado.** El alcance abarcó desde la segmentación de red y la gestión de permisos, hasta la protección contra vulnerabilidades de código y dependencias, dejando explícitamente fuera de alcance amenazas como atacantes dentro de la red privada.

## 2. Arquitectura del sistema en breve

Para quien no conoce la infraestructura: el sistema vive en cuatro VMs sobre una misma red privada, más un host físico adicional. Un host ejecuta `cloudflared`, el único componente con salida a Internet; un segundo clúster k3s corre el pipeline de CI (construcción, escaneo y publicación de imágenes); un tercer clúster k3s aloja la aplicación y todas sus bases de datos; un registro de imágenes (Harbor) y un almacén de objetos para backups (MinIO) corren aparte, este último en un host físico distinto, para que un fallo del host principal no se lleve también las copias de seguridad.

Dentro del clúster de aplicación, el sistema se organiza en **tres niveles de confianza**, cada uno en su propio *namespace* de Kubernetes (la unidad de Kubernetes que agrupa procesos relacionados y sirve de frontera para políticas de red, cuotas y permisos) con su propia base de datos:

| Nivel       | Namespace                       | Contenido                               | Postgres/Redis          |
|-------------|---------------------------------|-----------------------------------------|-------------------------|
| Crítico     | buscaviejos-main                | rama main, producción                   | dedicados               |
| Persistente | buscaviejos-develop / release/* | ambientes de integración estables       | compartidos             |
| Preview     | preview-{sha7}                  | un ambiente efímero por cada PR abierto | compartidos, temporales |

Cada nivel corre con prioridad de planificación distinta, de modo que ante presión de memoria el clúster libera primero las *previews*, nunca producción. El flujo de una solicitud es: navegador → túnel saliente de Cloudflare → borde de Cloudflare (TLS terminado ahí) → *ingress controller* dentro del clúster → pod de frontend o backend → base de datos del nivel correspondiente. El despliegue es de tipo GitOps: un `git push` en los repos de aplicación dispara la construcción y el escaneo de la imagen en el clúster de CI; solo si eso pasa, se dispara un despliegue en el clúster de aplicación, que aplica los manifiestos de un repositorio de infraestructura separado, la única fuente de verdad de qué corre en el clúster.

## 3. Modelo de amenaza

Siguiendo el marco objetivo / atacantes / política / mecanismo, el **objetivo de seguridad**, en orden de prioridad, es: **integridad** del estado del juego (saldo, inventario, resultados, historial), **confidencialidad** de los datos de usuario y de las credenciales de infraestructura, y **disponibilidad** del servicio. El orden no es arbitrario: la economía y los resultados de partidas pierden todo su valor si pueden falsificarse; los datos de usuario que se manejan (nombre, IDs, mensajes) no tienen la sensibilidad de un sistema bancario o médico; una caída temporal es una molestia, no una falla crítica.

Los **atacantes considerados** son: (1) un usuario con cuenta válida que intenta hacer trampa, escalar privilegios o atacar a otros jugadores; (2) un atacante externo sin credenciales que explota una vulnerabilidad del servidor o de una dependencia de terceros; (3) un atacante de cadena de suministro, una dependencia o herramienta de CI/CD comprometida. Quedan **explícitamente fuera de alcance** un atacante que ya está dentro de la red privada del clúster (el tráfico interno entre el túnel y el balanceador, y entre servicios dentro del clúster, va sin cifrar, una decisión consciente y no un descuido) y el compromiso físico de las máquinas.

La política se traduce en mecanismos concretos:

| Política                                                        | Mecanismo                                                                   |
|-----------------------------------------------------------------|-----------------------------------------------------------------------------|
| Solo usuarios autenticados pueden jugar o chatear               | JWT verificado en cada solicitud HTTP y en el <i>handshake</i> de WebSocket |
| Solo administradores pueden banear o asignar roles              | RBAC de aplicación, matriz de permisos por rol                              |
| Un moderador nunca puede moderar a un administrador             | Regla fija de anti-escalamiento, no derivada de comparación de niveles      |
| Toda acción administrativa queda registrada                     | Auditoría en dos niveles (aplicación y API server del clúster)              |
| Los secretos de infraestructura nunca se guardan en texto plano | Cifrado de secretos antes de comitearlos a git                              |

Los **supuestos** del modelo son la integridad del navegador del usuario, la hostilidad de la red externa, y la confianza en servicios delegados (Cloudflare, Clerk). La **superficie de ataque** principal incluye los endpoints públicos, dependencias de terceros, el pipeline de CI/CD, el registro de imágenes y el API server de Kubernetes.

Quedan **explícitamente fuera de alcance**: un atacante dentro de la LAN privada (tráfico interno sin cifrar por diseño), el compromiso físico de hardware/VMs, y la seguridad interna de servicios delegados. Definir estos límites permite distinguir entre brechas reales de seguridad y decisiones de alcance deliberadas.

## 4. Mecanismos de seguridad implementados

La exposición de los mecanismos de seguridad sigue una progresión: parte desde la infraestructura y el aislamiento de recursos, para integrar posteriormente conceptos transversales, como el principio de menor privilegio y la defensa en profundidad aplicados directamente sobre la lógica de operación del sistema.

### 4.1 Separación de privilegios

Aplicamos el principio de mínimo privilegio en múltiples niveles, aunque su implementación reveló inconsistencias reales entre componentes.

A nivel de datos, implementamos el patrón “*monitor/worker*”: en producción (*main*), el backend se conecta a Postgres con un rol estrictamente acotado a datos (DML), mientras las migraciones usan un proceso separado con permisos de esquema (DDL). Verificamos empíricamente esta restricción constatando que comandos destructivos (`DROP TABLE`) desde el rol de aplicación fallan de inmediato por falta de permisos (`must be owner`), mientras que desde el nivel persistente sí se procesan.

A nivel de red, segmentamos los tiers usando **NetworkPolicy**. Los namespaces de infraestructura aplican una política de solo *ingress* para proteger las bases de datos, mientras que las aplicaciones usan solo *egress* bajo un modelo de “denegar por defecto”. Las aplicaciones en sí no tienen política de *ingress*, una cobertura parcial consciente para priorizar la protección del dato antes que del cómputo.

Finalmente, a nivel de identidad e infraestructura, limitamos los accesos con éxito usando ACLs granulares en Redis, cuentas robot dedicadas en Harbor y roles acotados en ARC. La brecha honesta es que componentes con gran poder destructivo, como el runner de despliegue, el CronJob de limpieza y los charts de Traefik y Velero, mantuvieron privilegios amplios de cluster-admin, revelando que la disciplina de acotar permisos no se aplicó de forma pareja a toda la infraestructura.

## 4.2 Infraestructura Segura: Aislamiento, TLS y Secretos

El sistema se protege a nivel de infraestructura mediante tres pilares. Primero, el aislamiento se orquesta a través de Kubernetes, que a nivel de nodo utiliza primitivas del kernel de Linux (*namespaces* para aislar procesos/red y *cgroups* para acotar memoria/CPU). A nivel lógico, asignamos cada uno de los tres *tiers* (main, persistente y preview) a su propio **namespace de Kubernetes**. Esto nos dio aislamiento de red, de identidad (credenciales separadas) y de recursos físicos: usamos `PriorityClass` y `ResourceQuota` para garantizar que, ante presión en el nodo, el clúster sacrifique los pods efímeros (*previews*) y nunca producción. La brecha consciente es que esta disciplina no bajó al nivel de proceso: los contenedores carecen de un `securityContext` restrictivo (no hay perfiles *seccomp* ni *capability dropping*, y corren como *root* por defecto).

Segundo, la exposición a Internet se limita mediante un **túnel saliente de Cloudflare**. Al no abrir puertos de entrada, anulamos escaneos externos. El tráfico se asegura con TLS terminado en el *Cloudflare-edge*, descartando peticiones de hostnames desconocidos antes de que toquen nuestra red. Es importante ser precisos con esta medida: esto implementa un control de superficie de ataque genuino, pero no una arquitectura “Zero Trust” estricta, ya que carece de políticas de autenticación por identidad antes de llegar al servicio. Adicionalmente, el tramo interno entre el túnel y el balanceador de carga, así como el tráfico entre servicios dentro del clúster, van sin cifrar sobre la red privada, una decisión aceptada conscientemente dentro de nuestro modelo de amenaza..

Tercero, la gestión de credenciales reemplaza los `Secret` estándar en base64 por **Sealed Secrets**, utilizando criptografía asimétrica para versionar secretos de forma segura en Git. Solo el controlador dentro del clúster posee la llave privada para descifrarlos, garantizando una inyección segura.

## 4.3 Redis: autenticación y ACL

La seguridad de Redis evolucionó desde no tener protección a un control detallado de accesos (ACL). Se eliminó el usuario general y se crearon perfiles específicos: cada servicio solo tiene los permisos mínimos necesarios para su función. Por ejemplo, el backend puede gestionar datos pero tiene prohibido ejecutar comandos administrativos peligrosos, y los procesos de limpieza están estrictamente limitados a sus propias áreas.

El archivo de ACL se monta en los pods en modo **solo lectura**, así que ni siquiera un backend comprometido puede reescribir su propia política de permisos desde dentro del contenedor. Sumado a esto, como Redis de los tiers persistente/preview es compartido entre varios ambientes, cada uno prefija sus llaves, el cual es un aislamiento lógico adicional independiente del ACL. El resultado son tres capas independientes protegiendo el mismo recurso: aislamiento de red (`NetworkPolicy`), autenticación (usuario+contraseña) y autorización por categoría de comando (ACL): el cual es un claro ejemplo de **defensa en profundidad** en el proyecto.

## 4.4 Protección contra XSS y seguridad web

La defensa contra *cross-site scripting* (XSS) y otras vulnerabilidades web emplea **defensa en profundidad**. En el backend, `Helmet` añade cabeceras seguras contra *MIME-sniffing*. En el proxy, una estricta Content Security Policy (CSP) bloquea *scripts inline* (vector clásico de XSS) y embebido ajeno vía `frame-ancestors 'none'` (mitigando *clickjacking*), con la única excepción controlada del CAPTCHA Turnstile. Finalmente, combinamos la sanitización de `dompurify` con el auto-escape

nativo de React para garantizar una renderización segura. De forma incidental, la aplicación resultó estructuralmente inmune al CSRF clásico al utilizar tokens explícitos de autorización en cabeceras HTTP, descartando el envío automático de cookies de sesión.

#### 4.5 Detección de vulnerabilidades y CVEs

Trivy corre en el pipeline de CI sobre cada imagen construida en `main/release/*`, con `severity: CRITICAL, HIGH` y `exit-code: '1'`: si encuentra una CVE crítica o alta, el build falla ahí mismo y la imagen nunca llega al registro ni al clúster: bloqueando completamente el deploy. Antes de los arreglos motivados por este proyecto se detectaron y cerraron diecisiete vulnerabilidades reales (12 frontend y 5 backend) en dependencias de terceros (`js-cookie`, `ws`, `react-router`, `vite`, `qs`, `uuid`, etc). Entre ellas destacan la **CVE-2026-48779** en `ws` y la **CVE-2026-46625** en `js-cookie`, las cuales fueron remediadas fijando versiones parchadas mediante `pnpm.overrides`, dejando registro auditable en los commits. Además, configuramos el sistema para anclar estrictamente estas versiones revisadas, previniendo regresiones.

También decidimos eliminar `npm` y `npx` de la imagen final del backend. Esto no fue una medida preventiva general, sino una respuesta específica a la CVE-2026-33671, que afectaba a `npm` y `corepack` en nuestra imagen base (`node:22-alpine`). Como estos binarios no son necesarios en producción, decidimos borrarlos directamente, reduciendo de paso la superficie para ataques estilo “*living off the land*”. El siguiente fragmento de nuestro `Dockerfile` ilustra la eliminación agresiva de estas herramientas una vez finalizada la instalación de dependencias:

```
RUN pnpm install --frozen-lockfile --prod && \
    rm -rf /root/.cache \
        /usr/local/lib/node_modules/npm \
        /usr/local/bin/npm \
        /usr/local/bin/npx
```

Quedan brechas honestas: Harbor (el registro propio) tiene el escaneo de Trivy al hacer *pull* deshabilitado (`WITH_TRIVY=False`), el control real está solo en el *push* desde CI, lo cual es defendible (bloquea antes de que la imagen exista en cualquier lugar) pero deja sin cubrir una imagen que llegue al registro por otra vía.

#### 4.6 Disponibilidad y Resiliencia

La disponibilidad, el último de nuestro modelo de amenaza pero no menos importante, se asegura combinando controles de tráfico, orquestación dinámica y estrategias de recuperación ante desastres:

- **Protección de tráfico y balanceo:** Traefik aplica *rate limiting* (120 req/min) en los ambientes de producción (`main`) e integración (`develop`) para mitigar abusos a la API. Además, la correcta configuración del balanceador resultó crucial para la estabilidad: el primer día en producción multiréplica, una condición de carrera entre el *handshake* de *polling* y el *upgrade* a WebSocket causaba rechazos constantes. Aunque la reacción inicial fue bajar a una sola réplica (sacrificando resiliencia para recuperar correctitud), la solución definitiva fue implementar un `IngressRoute` con sesión pegajosa por cookie (`httpOnly`, `secure`), garantizando la afinidad de sesión del cliente sin perder la capacidad de absorber ataques volumétricos.
- **Reducción de superficie y ocultamiento:** Implementamos KEDA para escalar a cero los pods de los ambientes *preview* tras 20 minutos de inactividad; esto no solo ahorra cómputo, sino que “apaga” vectores de ataque cuando el ambiente no se está usando. Adicionalmente, el manejador de errores centralizado del backend intercepta cualquier fallo no controlado y envía un mensaje genérico, cumpliendo con las directrices de OWASP al evitar la fuga de *stack traces* o configuración interna hacia un posible atacante.

- **Recuperación ante desastres:** Utilizamos Velero para respaldar de forma automática y diaria los datos críticos hacia MinIO, el cual se ubica en un host físico independiente (VM4) para aislar el riesgo de una falla catastrófica de hardware en el servidor principal. La importancia de este control se demostró empíricamente durante un incidente grave: un CronJob de limpieza, que operaba con permisos excesivos y sufrió un error de *label*, borró accidentalmente el *namespace* de infraestructura compartida que alojaba las bases de datos de 34 ambientes simultáneos, validando que los respaldos comprobados son la última barrera real de defensa.

#### 4.7 Modelo de guardia

El modelo de guardia centraliza la autenticación, autorización y auditoría de cada solicitud a recursos protegidos. La autenticación se verifica globalmente mediante JWT (vía Clerk) tanto en HTTP como en el *handshake* de WebSockets, evitando el acceso a canales sin credenciales validadas. La autorización se gestiona mediante middlewares en cascada (`requireAuth` → `requireUser` → `requirePermission`), lo cual centraliza la validación de acceso y elimina la responsabilidad de verificación explícita en cada controlador.

Sobre esta base, opera un sistema de RBAC propio con 17 permisos organizados en tres niveles (`user`, `mod`, `admin`) y una matriz de control de acceso explícita. La función `hasPermission` permite restricciones individuales con expiración, permitiendo revocar permisos temporalmente sin alterar el rol global. La función `canModerate` actúa como defensa contra el escalamiento de privilegios, estableciendo reglas fijas, como `if (targetRole === "admin") return false`, que impiden que incluso usuarios con alto rango puedan moderar a otros administradores. Adicionalmente, existe un eje de RBAC a nivel de recurso, permitiendo que un usuario tenga autoridad específica sobre un recurso (como ser dueño de un canal) sin elevar su jerarquía global.

La auditoría se implementa a dos niveles. A nivel de aplicación, `auditLog.ts` serializa eventos sensibles a JSON sobre `stdout`, capturados por el pipeline Promtail-Loki y centralizados en un dashboard de Grafana. A nivel de Kubernetes, la política de auditoría de k3s aplica tres niveles de detalle según la sensibilidad del recurso (desde `Metadata` para secretos hasta `RequestResponse` para ejecuciones de comando). Esta configuración permite registrar actores, objetivos y cambios de estado con alta fidelidad, verificada en vivo ante accesos a recursos críticos. A continuación se presenta un evento real capturado por la auditoría, demostrando cómo k3s delata qué servicio intenta acceder a credenciales internas:

```
{
  "level": "Metadata",
  "verb": "watch",
  "user": {
    "username": "system:serviceaccount:monitoring:kube-prom...grafana"
  },
  "sourceIPs": ["10.42.0.82"],
  "objectRef": {
    "resource": "secrets",
    "namespace": "buscaviejos-main"
  }
}
```

Otro componente clave es el *Active Session Eviction*: ante acciones críticas como `baneos`, `limitUserSockets` utiliza el adaptador de `Socket.IO` en Redis para forzar la desconexión inmediata de todas las sesiones activas del usuario, independientemente de la réplica en la que estén conectadas. Por último, una limitación honesta de este modelo es que el log de auditoría escribe a `stdout` sin persistencia propia; su durabilidad depende estrictamente de que el pipeline de observabilidad externo capture y retenga la información antes de que los pods roten o se reinicien.

## 4.8 Supply chain

La seguridad de la cadena de suministro asume que la confianza en dependencias transitivas es una superficie de ataque crítica. Utilizamos `pnpm-lock.yaml` con `--frozen-lockfile` y `pnpm.overrides` para anclar versiones exactas, previniendo actualizaciones silenciosas maliciosas en sub-dependencias. Sin embargo, reconocemos brechas residuales difíciles de cerrar: carecemos de un SBOM formal, Harbor tiene apagada la verificación de firmas criptográficas (Cosign), y nuestras GitHub Actions usan tags mutables de versión semántica en vez de *pinning* por SHA, lo que deja teóricamente expuesta la cadena de CI a ataques de reescritura.

## 5. Metodología y desarrollo

El trabajo se organizó en dos frentes paralelos: la implementación incremental de cada una de las seis medidas prometidas sobre el clúster real, y, para este informe en particular, una auditoría de verificación: en vez de aceptar la documentación interna del proyecto como fuente de verdad, cada afirmación relevante se intentó confirmar contra el estado vivo del clúster (`kubectl`, lectura directa de manifiestos, lectura del código fuente de ambos repos) o contra el historial real de commits. Esta disciplina de “verificar contra el sistema en vivo, no contra lo que la documentación dice” surgió, en parte, de un error real cometido durante el propio proceso: una primera verificación de la sesión pegajosa de Socket.IO consultó únicamente el recurso Ingress estándar de Kubernetes y concluyó, incorrectamente, que la mitigación no existía, cuando en realidad estaba implementada como un `IngressRoute`, un *Custom Resource Definition* propio de Traefik que `kubectl get ingress` no lista. El error no fue inventar evidencia, fue verificar el tipo de recurso equivocado en un sistema con mecanismos de enrutamiento superpuestos. Corregirlo, y dejarlo documentado como ejemplo de un riesgo real al auditar sistemas complejos, terminó siendo, en sí mismo, parte del aprendizaje metodológico del proyecto.

Las herramientas principales fueron: `kubectl`/SSH para inspección y verificación en vivo del clúster; Helm para instalar y gestionar componentes de terceros (Sealed Secrets, KEDA, Velero, ARC, el stack de observabilidad Prometheus/Loki/Grafana); Trivy como escáner de CVEs integrado al pipeline; GitHub Actions como motor de CI/CD; Sequelize como ORM (que, como efecto colateral deseable, parametriza automáticamente casi todas las consultas SQL, cerrando por diseño la categoría de inyección SQL del Top 10 de OWASP); y Clerk como proveedor de identidad delegado.

Entre las decisiones de diseño más relevantes está delegar la autenticación a Clerk. Es un *trade-off* consciente: sacrificamos control sobre la infraestructura identitaria para mitigar de raíz los riesgos históricos asociados a implementaciones manuales (almacenamiento seguro, *hashing* costoso, manejo de MFA).

Respecto a la propuesta original, los cambios más notables fueron la evolución progresiva de la seguridad en Redis (hasta ACL granular) y la consolidación del “modelo de guardia”, que pasó de ser una promesa vaga a ser el control más robusto. El desafío más complejo fue operativo: llevar el sistema a producción multiréplica expuso de golpe varias fallas de sincronía y derivó en el incidente con el CronJob que borró bases de datos activas.

## 6. Resultados

Las seis medidas prometidas se entregaron, con matices honestos: separación de privilegios profunda (con el caso del rol de solo-lectura/escritura de datos frente al rol dueño del esquema como ejemplo más citable, y la inconsistencia entre niveles como hallazgo honesto); detección de CVEs sólida como condición de bloqueo en el pipeline, con cobertura asimétrica entre repositorios y entre construcción/descarga de imágenes; autenticación y control de acceso de Redis con la historia de iteración más completa del proyecto; defensa contra XSS más amplia que lo prometido, cubriendo de paso *clickjacking*, *MIME-sniffing* y, de forma incidental, CSRF; túnel con TLS entregado como eliminación de superficie de entrada,

sin el componente de autenticación por identidad; y modelo de guardia, el ítem más completo, con auditoría verificable en dos niveles distintos.

Más allá de lo prometido, el proyecto entregó: un *pooler* de conexiones de base de datos como capa adicional de aislamiento, límite de tasa de solicitudes a nivel de borde, la política de auditoría nativa del API server del clúster, backups automatizados en un host físico separado, escalado a cero de ambientes de *preview* inactivos, cifrado de credenciales antes de comitearlas a git, el sistema de control de acceso por roles de diecisiete permisos, y un manejador de errores centralizado que nunca expone detalles internos al cliente.

La evidencia detrás de estas afirmaciones no es documental sino verificada en vivo: respuestas NOAUTH/NOPerm reales contra el Redis en producción, errores de permiso reales (`must be owner of table users`) contra el Postgres de `buscaviejos-main`, una entrada real del *audit log* de k3s capturando un acceso a `secrets`, y la confirmación de que el `IngressRoute` de sesión pegajosa está efectivamente desplegado y activo. Frente al modelo de amenaza de la sección 3, las amenazas más reducidas son: el abuso de comandos administrativos sobre Redis y Postgres por parte de un backend comprometido (gracias a los roles de mínimo privilegio), el robo de credenciales desde el repositorio de infraestructura (gracias a `Sealed Secrets`), la ejecución de scripts maliciosos inyectados vía contenido de usuario (gracias a `CSP+DOMPurify+React`), y la propagación silenciosa de dependencias vulnerables (gracias al *gate* de Trivy). Las amenazas que **no** se reducen de forma significativa son: el compromiso de un componente con `cluster-admin` (el daño potencial sigue siendo total control del clúster), y un atacante que ya logró ejecución de código dentro de un contenedor (sin *capability dropping* ni `seccomp`, el contenedor no limita mucho más allá de lo que ya limita el propio proceso de la aplicación).

## 7. Seguridad del CI/CD y Riesgos residuales (Trabajo a Futuro)

El pipeline de CI/CD utiliza Actions Runner Controller (ARC) con runners efímeros. Aunque el diseño por defecto de ARC es altamente restrictivo, el equipo optó por asignar permisos de `cluster-admin` al workflow de despliegue para facilitar la ejecución de `kubectl apply`, revirtiendo un esquema originalmente seguro y exponiendo la totalidad del clúster ante un compromiso del runner.

Este caso evidencia una falta de disciplina en el componente más crítico del pipeline, especialmente al contrastarlo con el control plane de ARC o el diseño de otros componentes donde sí se aplicaron RBACs correctamente acotados. Esta brecha operativa, que contrasta con el control aplicado en otras áreas del clúster, ilustra los retos de mantener una estricta separación de privilegios en componentes críticos de la infraestructura.

La siguiente tabla resume las brechas abiertas más relevantes, ya priorizadas por severidad, con su esfuerzo de cierre estimado:

| Brecha                                                                                                                                                                                 | Severidad | Esfuerzo de cierre                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|------------------------------------------|
| Sin Pod Security Context: contenedores corren como root, sin <i>capability dropping</i> ni perfil <code>seccomp</code> ; agravado porque los Dockerfiles no declaran <code>USER</code> | Alta      | Bajo                                     |
| CronJob de limpieza de previews con <code>cluster-admin</code> (solo necesita 3 verbos acotados)                                                                                       | Alta      | Bajo (RBAC acotado ya diseñado)          |
| Runner de CD con <code>cluster-admin</code>                                                                                                                                            | Media     | Difícil (requiere refactor del pipeline) |

| Brecha                                                                                                        | Severidad  | Esfuerzo de cierre                                     |
|---------------------------------------------------------------------------------------------------------------|------------|--------------------------------------------------------|
| Traefik y Velero también con <code>cluster-admin</code> (patrón común al instalar charts de Helm de terceros) | Baja-Media | Medio                                                  |
| NetworkPolicy de ingress incompleta en namespaces de aplicación                                               | Media      | Bajo                                                   |
| Harbor no escanea CVEs al hacer <i>pull</i>                                                                   | Media      | Bajo                                                   |
| Trivy no corre en develop en el backend (sí en el frontend)                                                   | Media      | Bajo                                                   |
| Sin rate limiting por evento de Socket.IO (solo a nivel HTTP)                                                 | Media      | Bajo                                                   |
| Rol de Postgres del tier persistente es dueño del esquema, no DML-only                                        | Media      | Bajo-Medio                                             |
| Logs de auditoría de aplicación sin garantía de durabilidad propia (dependen de Loki)                         | Baja-Media | Medio                                                  |
| Sin SBOM ni firma de imágenes (Cosign disponible en Harbor, no activado)                                      | Media      | Medio                                                  |
| GitHub Actions sin <i>pinning</i> por SHA, incluida la propia herramienta de escaneo                          | Baja       | Bajo                                                   |
| Consola de administración de MinIO expuesta en LAN (puerto 9001)                                              | Baja       | Bajo                                                   |
| Tráfico LAN interno sin cifrar                                                                                | Baja       | Aceptada conscientemente (fuera del modelo de amenaza) |

Estas brechas no debilitan el proyecto; son, en rigor, el contenido que el criterio de evaluación de “profundidad técnica” busca: mostrar que se conocen, se priorizaron, y que algunas se dejaron abiertas conscientemente (como el tráfico LAN sin cifrar) en vez de afirmar que el sistema es perfecto.

## 8. Discusión

Las decisiones del proyecto priorizan el riesgo de forma razonada. La asimetría en las NetworkPolicy protege los datos con rigor pero omite políticas de ingress en la aplicación, dejando una brecha potencial para la comunicación directa entre pods sin pasar por Traefik. Asimismo, el tráfico LAN sin cifrar se considera aceptable al no ser accesible desde Internet, aunque esta seguridad depende de la topología de red actual y no de controles técnicos internos adicionales.

La ventaja más clara del proyecto frente a un ejercicio de curso típico es que toda la evidencia es verificable contra un sistema real, en producción, con incidentes reales ya ocurridos, no hipótesis de seguridad sin probar. Eso tiene un costo: la complejidad operacional de mantener tres tiers, múltiples bases de datos compartidas, y un pipeline de CI/CD propio introduce más superficie de configuración que un sistema más simple, y de hecho varias de las brechas documentadas (los `cluster-admin` de Traefik/Velero, la falta de `securityContext`) son efectos secundarios de instalar componentes de terceros “tal como

vienen” para acelerar el desarrollo, un *tradeoff* común y razonable, pero que acumula deuda de seguridad si no se revisa después.

El riesgo residual más serio, si se tuviera que elegir uno, es la combinación de los cinco `ClusterRoleBinding` a `cluster-admin` con la ausencia de `securityContext` en los pods: un atacante que comprometa el runner de CD o el CronJob de limpieza (ambos con acceso total al clúster) no enfrenta ninguna fricción adicional dentro del contenedor mismo (sin *capability dropping*, sin usuario no-root, sin perfil *seccomp*) para intentar salir de él o escalar más. Son dos brechas independientes que, combinadas, son más graves que cualquiera por separado: la primera da el motivo para querer comprometer ese componente, la segunda hace que comprometerlo sea más fácil de lo necesario.

## 9. Conclusiones

Este proyecto aplicó, sobre un sistema real en producción y no sobre un laboratorio aislado, un conjunto amplio de conceptos centrales del curso: separación de privilegios en la línea de OKWS y de la arquitectura de seguridad de Google (el patrón monitor/worker en Postgres, los roles acotados de Redis); aislamiento mediante namespaces de Kubernetes como capa de orquestación sobre los namespaces de Linux, con la brecha honesta de no haber bajado al nivel de *capabilities/seccomp* dentro del pod; el modelo de guardia (autenticar/autorizar/auditar) implementado de forma completa; defensa en profundidad aplicada de forma concreta en al menos dos subsistemas (Redis, defensa XSS); y seguridad de cadena de suministro, y decisiones reales del proyecto.

El resultado se resume en tres aprendizajes clave. Primero, para operaciones destructivas, **un solo control es insuficiente**; el incidente en el namespace de infraestructura demostró que la defensa en profundidad es vital ante fallos reales. Segundo, **la eficiencia suele comprometer la seguridad**: priorizar atajos sobre permisos acotados transformó un error de configuración en un impacto sobre treinta y cuatro ambientes. Tercero, **la falta de una nube administrada obliga a una verificación rigurosa** contra el sistema en vivo, no solo contra la documentación. Este último aprendizaje fue de los más importante y no fue técnico en el sentido estrecho, sino metodológico: la seguridad de un sistema real se mide contra lo que el sistema *hace* cuando se le consulta directamente, no contra lo que la documentación dice que hace. Varios de los hallazgos más valiosos de este informe solo aparecieron al verificar en vivo contra el clúster. Eso es, en sí mismo, una instancia concreta de una idea central del curso: los modelos de amenaza y las suposiciones de seguridad fallan con más frecuencia por desactualización o por verificación superficial que por errores de diseño profundos, y la disciplina de re-verificar contra el sistema real, no contra lo que se asume sobre él, es, en la práctica, tan importante como cualquier mecanismo individual descrito en este informe.

Finalmente, las brechas y riesgos residuales sistematizados en los resultados (Sección 6) delinean de forma natural nuestro **trabajo a futuro**. Los siguientes pasos lógicos para fortalecer el clúster consistirían en bajar el aislamiento al nivel de proceso mediante *Pod Security Contexts* (restringiendo contenedores *root*, descartando *capabilities* y aplicando perfiles *seccomp*), refactorizar el pipeline de CD para eliminar los privilegios globales (`cluster-admin`) del *runner*, y sellar la comunicación interna a través de una política de *ingress* estricta en los namespaces de aplicación.

## 10. Material del Proyecto

El código fuente de la aplicación, los manifiestos de infraestructura y el material complementario se encuentran disponibles para su revisión en la siguiente carpeta:

- **Carpeta de Material del Proyecto (OneDrive)**